CityMind AI — 5-Minute Master Defense & Codebase Navigation Manual

Project Review Preparation & Comprehensive Defense Document

Frameworks: Eclipse SUMO, TraCI, PyTorch, Stable-Baselines3

Scenario: Ingolstadt InTAS (98 Signalized Junctions)

Reference Book: Sudharsan Ravichandiran (Packt, 2018)

CityMind AI — 5-Minute Master Defense & Codebase Navigation Manual

---

Part 1: The 5-Minute Speaking Script (Timed for Perfection)

Deliver this script smoothly in **exactly 5 minutes**. It addresses every previous question and demonstrates deep theoretical mastery from **Sudharsan Ravichandiran's textbook** alongside rigorous software engineering.

```
      5-MINUTE ORAL DEFENSE TIMELINE
0:00 ─── [1] The Problem & Real-World InTAS Scenario (Ingolstadt)
1:00 ─── [2] Simulator & Middleware: Why SUMO & What is TraCI?
2:00 ─── [3] RL Formulation: Why PPO, IPPO Architecture & Bellman Principles
3:00 ─── [4] The Breakthroughs: Split-Clipping, 300m Feeders & Delay-Level Reward
4:00 ─── [5] Evaluation: Drain Metrics, Survivor Bias & 10/10 Seed Victory
5:00 ─── Conclusion & Ready for Q&A
```

Minute 1:00 — Problem Statement & The Real-World InTAS Scenario

> *"Good morning, Professor. Today I am presenting **CityMind AI**, a decentralized Deep Reinforcement Learning system for city-wide traffic signal control.*

>

> *Most academic RL papers test on synthetic, 4-way toy grids with uncalibrated fixed-time timers and claim massive 20% improvements. We evaluated whether RL can outperform real-world civil engineering on a genuine city scale: the **Ingolstadt InTAS network** in Germany, consisting of **98 heterogeneous, signalized intersections** during the morning rush hour (07:00 to 08:00) with over **15,000 scheduled trips**.*

>

> *Our benchmark target is not an artificial baseline, but **SUMO's calibrated vehicle-actuated controller**—the actual industry standard running embedded induction loops with dynamic gap-out logic."*

Minute 2:00 — Simulation Engine & Middleware: Why SUMO & TraCI?

> *"To simulate microscopic traffic physics, we use **Eclipse SUMO** (sumo_session.py). SUMO models discrete-time, continuous-space driver dynamics including Krauss car-following, lane changing, and intersection clearance.*

>

> *To connect our Python RL agent to SUMO, we use **TraCI (Traffic Control Interface)** (sumo_connection.py). TraCI is a TCP socket client-server protocol enabling bidirectional interaction: the simulation acts as the server, and our Python environment acts as the client.*

>

> *In our codebase, TraCI functions in two ways:*

> *1. **Sensor Ingestion:** At each 5-second decision step, we query vehicle queues, halted cars, and downstream speeds via `traci.lane.getLastStepHaltingNumber` and `traci.simulation.getPendingVehicles` (lane_stats.py, local_reward.py).*

> *2. **Actuation:** After filtering RL decisions through our deterministic SafetyLayer, we command signal phases using `traci.trafficlight.setPhase` and advance time using `traci.simulationStep()`.*

>

> *For maximum training throughput, we wrap this via `libsumo` —a direct C++ SWIG binding eliminating socket serialization overhead."*

Minute 3:00 — RL Formulation: Why PPO, IPPO Architecture & Textbook Theory

> *"Now addressing the core RL theory from **Sudharsan Ravichandiran's book** (specifically **Chapter 1 on Dec-POMDPs, Chapter 10 on Actor-Critic, and Chapter 11 on PPO**):*

>

> ***Why not DQN (Chapter 8)?** Centralized DQN suffers from the curse of dimensionality: with 98 intersections, the joint discrete action space is $2^{98} \approx 3.16 \times 10^{29}$ actions per step, making Q-value convergence mathematically intractable.*

> **Why not vanilla Policy Gradients / REINFORCE (Chapter 11)?** REINFORCE has high variance and poor sample efficiency because it relies on full Monte Carlo returns.

> ***Why PPO (Chapter 11, p. 245)?** PPO achieves trust-region stability using a first-order **Clipped Surrogate Objective**:*

> *$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)\hat{A}_t, \, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]$$*

> *with Generalized Advantage Estimation (GAE, $\gamma=0.99, \lambda=0.95$). This prevents destructive policy collapse while running standard first-order Adam optimization.*

>

> **How we structured PPO:** We formulate the city as a **Decentralized Partially Observable MDP (Dec-POMDP)** using **IPPO (Independent PPO with Parameter Sharing)** (ippo_agent.py). All 98 intersections have their own decentralized observations and rewards, but share **one single neural network policy** ($22 \rightarrow 256 \rightarrow 256 \rightarrow 2$, with Tanh activations). Each simulation step generates 98 decentralized transitions, providing massive sample efficiency across 6 parallel SUMO instances (588 parallel environments)."

Minute 4:00 — The Key Engineering Breakthroughs That Beat the Baseline

> "Early in the project, standard PPO failed against SUMO actuated. We analyzed the failures and engineered 5 major breakthroughs:
>
> 1. **Separate Actor/Critic Gradient Clipping** (split_clip.py): In standard Stable-Baselines3, actor and critic gradients are clipped together under a single norm (0.5). The critic's gradient norm was **3,830** while the actor's was **0.19**. Joint clipping scaled the actor's gradients down by $7,600\times$, below Adam's epsilon threshold, causing policy entropy to freeze at 90%! By clipping actor and critic parameters independently at 0.5, policy update speed increased by $10\times$.
>
> 2. **300-Meter Feeder Lane Queues** (local_observation.py): In urban networks, stop-line lanes can be as short as 20 meters (holding only 2 cars). If an agent only checks stop-lines, a 60-car queue directly behind it remains invisible. We recursively walk upstream roads up to 300 meters to measure true incoming queues.
>
> 3. **Delay-Level Reward Design** (local_reward.py): We abandoned naive 'change in waiting time' (ΔW), which agents game by rapidly oscillating lights. Instead, our reward is a direct cost of stopped vehicles, downstream spillback risk, switching penalty, and outside-network backlog:
>
> $$r_i(t) = -0.1 \times \min\left(Q_i + 2D_i + S_i + 0.5P_i + 0.5\frac{P_{total}}{98}, \, 200 \right)$$
>
> 4. **Outside-Network Pending Demand** (local_observation.py): Cars queued outside city entry edges are fed into the observation (feature 16) and reward, preventing the agent from blocking entry gates.
>
> 5. **Deterministic Safety Layer** (safety.py): Guarantees physical clearance (Yellow + All-Red) and bounds green phases between minimum and maximum constraints."

Minute 5:00 — Metrics, Experimental Protocol & 10-Seed Victory

> "To evaluate performance, we developed the **Drain Metrics Protocol** (drain_metrics.py). Standard metrics suffer from **Survivor Bias**: if an agent causes gridlock on side streets, only a few lucky highway cars finish, making 'mean wait of completed cars' look falsely positive. Drain metrics score **all scheduled vehicles** against theoretical free-flow completion.

>

> We evaluated over **10 independent simulation seeds** (40 full 1-hour runs, paired seed-by-seed, detailed in docs/TEST_REPORT_10_SEEDS.md):

> ***Lag per scheduled car:*** *Reduced from $276.8|text{ s}$ to* ***$235.4|text{ s}$*** *($-41.4|text{ s}$,* ***won 10/10 seeds****, $p=0.002$).*

> **Trip time loss:** Reduced from 336.6 s to **286.4 s** (-50.2 s, **won 10/10 seeds**, $p=0.002$).

> ***Mean waiting time:*** *Dropped from $6.39|text{ s}$ to* ***$3.77|text{ s}$*** *(****$41.1|%$ reduction****,* ***won 10/10 seeds****).*

> **Trips completed:** Increased by **$+258$ vehicles** (**won 10/10 seeds**).

> ***City Backlog at 08:00:*** *Dropped by* ***-256 trapped cars*** *(****won 10/10 seeds****).*

>

> *In summary, CityMind AI demonstrates that decentralized, parameter-shared PPO with split gradient clipping and feeder-aware perception beats calibrated civil-engineering actuation across an entire city network. Thank you, and I look forward to your questions."*

Part 2: Deep Dive Q&A Defense (Professor's Previous Questions)

Use these targeted responses if the professor asks for details on specific areas.

```
                CORE Q&A MATRIX
┌──────────────────────────────┬──────────────────────────────────────────┐
│ Question                     │ Core Theoretical & Engineering Answer    │
├──────────────────────────────┼──────────────────────────────────────────┤
│ 1. What is TraCI? Where used?  │ TCP socket client-server; sensor/control │
│ 2. Why PPO over DQN/A2C/TRPO?  │ Continuous clipping, Dec-POMDP stability │
│ 3. How does PPO work here?     │ Parameter-shared IPPO, GAE, clipped loss │
│ 4. Why 22-dim MLP 256x256?     │ Spatial invariance + feeder awareness    │
│ 5. Why Split Gradient Clipping? │ Critic grad (3830) suppressed actor (0.19)│
│ 6. What are Drain Metrics?     │ Fixes survivor bias via scheduled demand │
└──────────────────────────────┴──────────────────────────────────────────┘
```

Q1: "What is TraCI? Why have you used SUMO? Where have you used TraCI in the code and how does it function?"

1. What is TraCI & Why SUMO?

- **SUMO (Simulation of Urban MObility):** An open-source, continuous-space microscopic traffic simulator. Each vehicle is simulated individually using car-following models (Krauss) and lane-changing models (LC2013). Toy grid environments cannot capture real-world traffic dynamics such as queue spillback, left-turn blocking, or acceleration delays.
- **TraCI (Traffic Control Interface):** The TCP-based network protocol enabling external applications to communicate with a running SUMO simulation process. SUMO runs as a socket server (listening on a TCP port), and our Python process connects as a socket client.

2. Where is it in the codebase?

- **Central Connection Hub:** `src/environment/sumo_connection.py` defines `_LazyTraci`. It dynamically loads `libsumo` (for speed during training) or fallback socket `traci`, ensuring the entire codebase imports from a single interface:

``` `python`

from src.environment.sumo_connection import traci, USING_LIBSUMO

```

- **Process Lifecycle:** src/environment/sumo_session.py handles startup and teardown:

``` `python`

Line 118: Starts the SUMO simulation subprocess via TraCI

traci.start(cmd)

```

`

- **Simulation Step & Signal Actuation:** src/environment/decentralized_sumo_env.py :

`python

```
Step 1: Query simulation time
sim_time = traci.simulation.getTime()
Step 2: Apply safe phase actuation (calls traci.trafficlight.setPhase)
self.safety.step(sim_time, actions, self.step_length)
Step 3: Advance simulation by 1 second
traci.simulationStep()
Step 4: Collect microscopic metrics
self.metrics.tick()
```

`

- **Live Sensor Reads:**

– traci.lane.getLastStepHaltingNumber(lane) : Counts halted vehicles (lane_stats.py).

– traci.lane.getLinks(lane) : Discovers downstream topology (local_observation.py).

– traci.simulation.getPendingVehicles() & traci.vehicle.getRoute(vid) : Tracks cars queuing outside the city gates (local_reward.py).

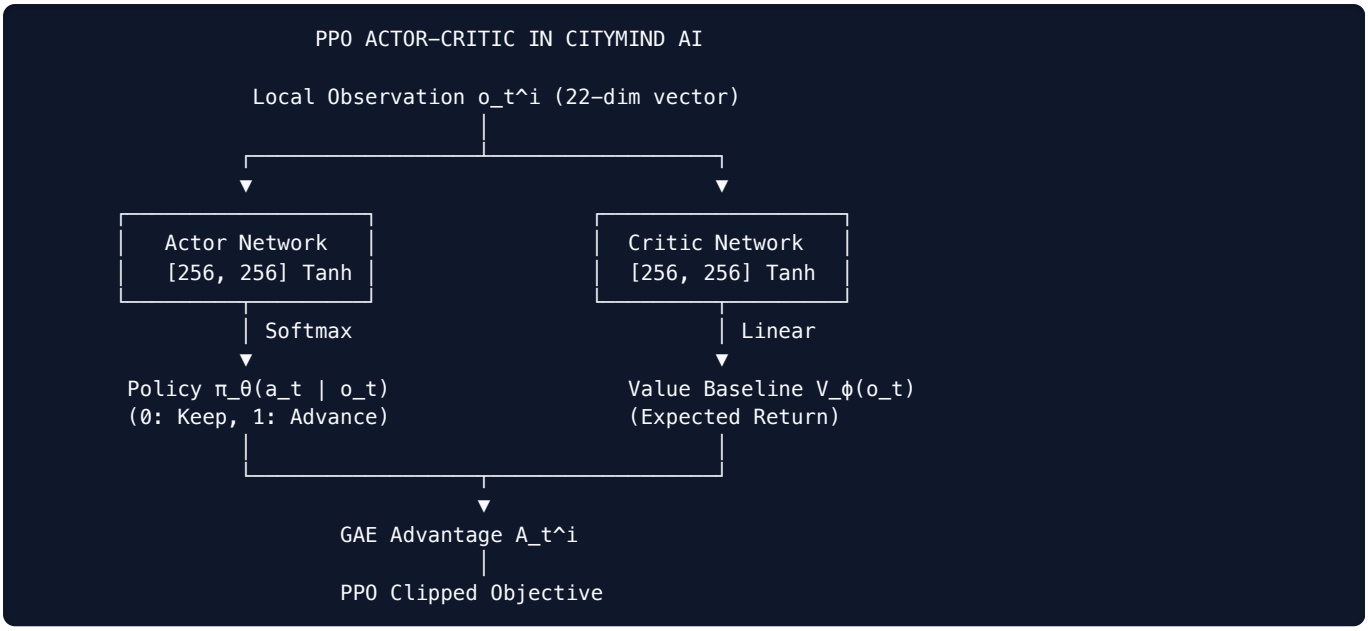
Q2: "Why PPO? Why not DQN, Q-Learning, REINFORCE, A2C, or TRPO? Connect this with Sudharsan Ravichandiran's book."

Grounded in **Ravichandiran (2018):**

Algorithm	Book Reference	Why it Fails on 98-Signal City Control	Why PPO Wins	
Q-Learning / Tabular TD	Ch 3 & Ch 5 (pp. 53–120)	Tabular state space is infinite (22×98 continuous values); cannot generalize across continuous state spaces.	PPO uses function approximation (neural networks) to map continuous states to actions.	
Deep Q-Networks (DQN)	Ch 8 (pp. 165–188)	Discrete action space for 98 signals is $2^{98} \approx 3.16 \times 10^{29}$. Q-learning over joint actions requires finding $\arg\max_a Q(s,a)$, which is NP-hard at this scale. In decentralized DQN, off-policy non-stationarity breaks the replay buffer.	PPO is on-policy and handles multi-agent decentralized policies naturally without replay buffer corruption.	
REINFORCE (Policy Gradient)	Ch 11 (pp. 226–230)	Uses full episode Monte Carlo returns $\sum \gamma^t R_t$. High variance, slow convergence, and no value function baseline.	PPO uses an Actor-Critic architecture with Generalized Advantage Estimation (GAE) to drastically reduce variance.	
A2C / A3C	Ch 10 (pp. 208–224)	Unconstrained policy updates. A single bad batch gradient can update θ too far, causing policy collapse (entropy drops or explodes, performance never recovers).	PPO clips the probability ratio $r_t(\theta)$ to $[1-\epsilon, 1+\epsilon]$, ensuring stable monotonic improvement.	
TRPO (Trust Region Policy Opt.)	Ch 11 (pp. 240–244)	Enforces $D_{KL}(\pi_{old} \| \pi) \le \delta$. Requires computing second-order Fisher Information matrix and conjugate gradients, which is computationally prohibitive for high sample throughput.	PPO achieves the same trust-region safety using a first-order clipped surrogate loss that trains via standard Adam.	

Q3: "How is PPO working in this project? Explain the Actor, Critic, Loss function, and GAE."

From **Ravichandiran Chapter 10 & 11**, PPO is an **Actor-Critic architecture**:



1. **The Probability Ratio:**

s_t)}{\pi_{\theta_{old}}(a_t

2. **Clipped Surrogate Objective:**

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta) \hat{A}_t, \, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t \right) \right]$$

- Here $\epsilon = 0.2$ (configs/ippo.yaml).

- If Advantage $\hat{A}_t > 0$ (action was good), r_t is capped at $1 + \epsilon = 1.2$ to avoid excessive updates.

- If Advantage $\hat{A}_t < 0$ (action was bad), r_t is capped at $1 - \epsilon = 0.8$.

3. **Generalized Advantage Estimator (GAE) ($\gamma=0.99, \lambda=0.95$):**

$$\hat{A}_t = \sum_{l=0}^\infty (\gamma \lambda)^l \delta_{t+l}^V, \quad \text{where } \delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$\lambda=0.95$ smoothly balances bias (TD estimate) and variance (Monte Carlo rollout).

4. **Total PPO Loss Function (Ravichandiran p. 247):**

$$L_{total}(\theta, \phi) = - L^{CLIP}(\theta) + c_1 L^{VF}(\phi) - c_2 S[\pi_\theta]$$

- $L^{VF}(\phi) = \frac{1}{2} (V_\phi(s_t) - R_t)^2$ (Critic Value Loss, $c_1 = 0.5$).

- $S[\pi_\theta]$ is policy entropy encouraging exploration ($c_2 = 0.01$).

Q4: "Why Split Gradient Clipping? Why was this the critical bug fix?"

- **The File:** src/agents/split_clip.py

- **The Problem:** In standard reinforcement learning libraries (Stable-Baselines3), gradients are clipped by a single global norm across all network parameters:

$$\text{clip_grad_norm_}(\text{actor_params} \cup \text{critic_params}, \, \text{max_norm}=0.5)$$

- **What Happened During Training:**

– Traffic reward values produced large Temporal Difference errors in the early stages, causing the **critic's gradient norm to surge to ~3,830.**

– In contrast, the **actor's gradient norm was only ~0.19.**

– When Stable-Baselines3 computed the combined norm:

$$\text{Scale Factor} = \frac{0.5}{\sqrt{3830^2 + 0.19^2}} \approx \frac{0.5}{3830} \approx 1.3 \times 10^{-4}$$

– The actor's gradients were multiplied by 1.3×10^{-4}, shrinking its effective gradient to 0.000025—below Adam optimizer's ϵ threshold (10^{-8})!

– **Result:** The actor stopped learning entirely, policy entropy remained frozen at 90%, and lights switched almost randomly.

- **The Solution:** We implemented SplitClipPPO (split_clip.py), which splits parameter groups and clips them independently:

```python
n_actor = original(actor, max_norm) # clipped at 0.5
n_critic = original(critic, max_norm) # clipped at 0.5
```

This allowed the actor to learn at full capacity, accelerating policy progress by $10\times$.

Q5: "What is your Neural Network Architecture, Observation Space, and Action Space?"

1. Observation Space (22 numbers per signal, squashed to $[-1, 1]$)

Implemented in src/environment/local_observation.py :

Because intersections have varying numbers of lanes (4 to 19 lanes), feeding raw lane vectors is impossible without variable-sized graphs. We created a **fixed 22-dimensional feature vector** that is identical across all 98 intersections:

- **Indices 0–5:** Active green phase (one-hot encoding across up to 6 green phases).
- **Index 6:** in_transition flag (1.0 if Yellow or All-Red clearance is currently running).
- **Index 7:** elapsed_over_min – current green duration relative to its authored minimum green ($\min(t / G_{min}, 3) / 3$).
- **Index 8:** elapsed_over_max – current green duration relative to its maximum green ($\min(t / G_{max}, 1.5) / 1.5$).

- **Indices 9–14:** slot_demand_i — Varaiya pressure for each green phase: d_i
- **Index 15:** max_queue — halted vehicles on the worst approach (including 300m feeders).
- **Index 16:** pending — vehicles queued outside the network waiting to enter via this intersection.
- **Index 17:** n_green — number of valid green phases for this signal divided by 6.
- **Indices 18–19:** Upstream road neighbors' queues (2 strongest connected junctions).
- **Indices 20–21:** Downstream road neighbors' queues (2 strongest connected junctions).

2. Action Space (Discrete(2))

Implemented in src/environment/decentralized_sumo_env.py:

- 0: Keep current green phase.
- 1: Advance to the next sequential green phase.
- **Safety Decoupling:** The agent never directly sets lights in SUMO. The action is passed to SafetyLayer, which enforces minimum green, maximum green, and the junction's authentic yellow and all-red clearance intervals.

3. Neural Network Architecture

- **Policy Type:** Multi-Layer Perceptron (MlpPolicy via PyTorch).
- **Layers:** 2 hidden layers with 256 units each: [22] -> [256] -> [256] -> [2 actions].
- **Activation Function:** Tanh (preferred over ReLU in continuous-control PPO for smooth gradient propagation and bounded output dynamics).

Q6: "What are Drain Metrics? How did you get the results and why do standard metrics fail?"

- **The File:** src/evaluation/drain_metrics.py

- **The Survivor Bias Trap:**

In standard literature, researchers report "Mean Waiting Time of Completed Trips". If a bad RL policy causes massive gridlock, long-distance trips are trapped and never complete. Only short trips near highway exits finish. The average wait time of completed cars looks artificially low, disguising severe network failure!

- **The Drain Solution:**

We evaluate against the **full scheduled demand** ($\sim 15,000$ cars):

1. For every car, calculate its ideal zero-delay completion time:

$$\text{ideal} = \text{depart_time} + \text{free_flow_travel_time}$$

2. Compute **Lag per Scheduled Car:**

$$\text{Lag} = \frac{1}{N_{scheduled}} \sum_{k=1}^{N_{scheduled}} (\min(\text{actual_arrival}_k, T_{end}) - \min(\text{ideal}_k, T_{end}))$$

If a car never enters or gets stuck, its completion time is penalized at the simulation end time ($T_{end} = 08:00$).

- **Results Verification:**

Evaluated over **10 random seeds** using paired t-tests and non-parametric sign tests (docs/TEST_REPORT_10_SEEDS.md):

- **Lag:** Model A achieved **235.4 s** vs. Baseline **276.8 s** (-41.4 s per car, **10/10 seeds won**, $p=0.002$).

- **Time Loss: 286.4 s** vs. **336.6 s** (-50.2 s per car, **10/10 seeds won**, $p=0.002$).

- **Mean Waiting Time: 3.77 s** vs. **6.39 s** (**41% reduction, 10/10 seeds won**, $p=0.002$).

- **Trips Completed: 11,842** vs. **11,585** (**+258 additional finished journeys, 10/10 seeds won**, $p=0.002$).

Part 3: Codebase Navigation Map with Clickable Hook Links

Click any link below to jump directly to the relevant code:

```
                                CITYMIND AI CODEBASE TREE
configs/
  ├── ippo.yaml                          <-- PPO hyperparameters (lr, gamma, clip, split_clip)
  └── environment.yaml          <-- Simulation duration, safety bounds, reward weights
src/
  ├── agents/
  │     ├── ippo_agent.py               <-- PettingZoo-to-VecEnv wrapper & training harness
  │     ├── split_clip.py               <-- Independent Actor/Critic gradient norm clipping
  │     └── model_contract.py           <-- Input validation and artifact packaging
  ├── environment/
  │     ├── sumo_connection.py          <-- TraCI / libsumo lazy proxy & SUMO_HOME resolver
  │     ├── sumo_session.py             <-- SUMO subprocess command builder & lifecycle
  │     ├── decentralized_sumo_env.py <-- PettingZoo ParallelEnv (98 agents, shared policy)
  │     ├── local_observation.py        <-- 22-dim feature builder & 300m feeder walker
  │     ├── local_reward.py             <-- Delay-level reward & outside backlog tracking
  │     ├── safety.py                   <-- Deterministic clearance state machine
  │     └── lane_stats.py               <-- Fast memoized TraCI sensor batch reads
  ├── controllers/
  │     ├── sumo_actuated.py            <-- Industry baseline (calibrated gap-out logic)
  │     └── fixed_time.py               <-- Classical pre-timed Webster baseline
  ├── evaluation/
  │     ├── drain_metrics.py            <-- Survivor-bias-free demand scoring
  │     └── evaluate.py                 <-- Head-to-head multi-seed benchmark runner
scripts/
  ├── run_ippo_training.py                   <-- CLI entrypoint for training from scratch
  └── evaluate.py                            <-- CLI entrypoint for 10-seed evaluations
docs/
  ├── TEST_REPORT_10_SEEDS.md                <-- Comprehensive 10-seed statistical evaluation report
  └── LEARNING_GUIDE_ALL_POLICIES.md         <-- Architectural manual across all controllers
```

Part 4: Concept Mapping — Sudharsan Ravichandiran Book ↔ CityMind AI

Reference this table to ground code details in textbook terminology:

Textbook Concept (Ravichandiran 2018)	Book Chapter & Pages	Theoretical Meaning	CityMind AI Implementation Link	
Dec-POMDP vs. MDP	Chapter 1 (pp. 14–18)	Multi-agent partially observable environment where agents observe local state o_t^i rather than global state $\mathcal{S}_t$.	Defined in decentralized_sumo_env.py and local_observation.py.	
Discount Factor (γ) & Returns	Chapter 3 (pp. 58–63)	$G_t = \sum_{k=0}^\infty \gamma^k R_{t+k+1}$; balances immediate vs future traffic delay ($\gamma=0.99$).	Configured in configs/ippo.yaml.	
Actor-Critic Architecture	Chapter 10 (pp. 208–212)	Two networks: Actor outputs action probabilities $\pi_\theta(a\	s)$; Critic evaluates state value baseline $V_\phi(s)$.	Initialized in ippo_agent.py with `MlpPolicy` `[256, 256]`.
Advantage Function ($A(s,a)$)	Chapter 10 (pp. 209–211)	$A(s,a) = Q(s,a) - V(s)$; quantifies how much better an action is compared to the baseline expectation.	Computed using GAE ($\lambda=0.95$) in ippo_agent.py.	
Exploration via Entropy Bonus (S)	Chapter 10 (pp. 211, 247)	$-c_2 \sum \pi \log \pi$ added to loss to prevent premature policy collapse to deterministic suboptima.	Configured via ent_coef: `0.01` in configs/ippo.yaml.	
PPO Clipped Surrogate Objective	Chapter 11 (pp. 245–247)	Clamps probability ratio $r_t(\theta)$ to $[1-\epsilon, 1+\epsilon]$ to prevent destructively large policy updates.	Configured via clip_range: `0.2` in configs/ippo.yaml.	
Gradient Clipping for Stability	Chapter 11 (pp. 231–235)	Prevents gradient explosion; modified here to decouple Actor and Critic gradient norms.	Custom implementation in split_clip.py.	

Part 5: Rapid-Fire Pocket Reference (Pre-Defense Cheat Sheet)

Keep these numbers and facts fresh right before your presentation:

- **City & Scenario:** Ingolstadt (InTAS), Germany, **98 traffic lights,** morning peak (07:00–08:00, 3600 seconds), **~15,000 scheduled vehicles.**
- **Algorithm: IPPO (Independent PPO with Parameter Sharing),** 1 single neural network controlling all 98 signals.
- **Network Shape:** Actor and Critic each have 2 hidden layers of 256 units with **Tanh** activations: 22 -> 256 -> 256 -> 2 **.**
- **Decision Time-step:** Every **5 seconds** (720 decisions per hour)**.**
- **Observation: 22 features** (normalized to $[-1, 1]$), including current phase, relative green duration, 6-phase Varaiya demand, 300m feeder queues, outside pending cars, and 4 neighbor queues**.**
- **Action:** Discrete(2)` (0 = keep current phase, 1 = advance to next phase), verified and safe-guarded by SafetyLayer.
- **Training Volume: 10.58 Million timesteps**, 100 PPO updates, 105,840 samples per update across 6 parallel SUMO instances.
- **Key Metric: Lag per scheduled car** (Drain Metric): Our model achieved **235.4 s** vs. SUMO Actuated's **276.8 s** (**41.4 s saved per vehicle across the entire city, winning 10 out of 10 seeds**).